INGENIERÍA Y CALIDAD DE SOFTWARE

Testing de software y testing ágil

13 de octubre de 2026

1. Fundamentos, niveles, estrategias y ambientes

El testing busca identificar defectos cuya presencia se asume. La ausencia de hallazgos en una corrida no prueba que el producto carezca de defectos. **Verificar** es controlar si lo construido funciona de acuerdo con lo especificado, mientras que **validar** es comprobar si se está construyendo el producto adecuado para la necesidad del usuario. El testing contribuye al control de calidad, pero la calidad no se agrega al final ni depende solamente del equipo que prueba.

Los **niveles de prueba** conservan un orden lógico: primero las unitarias, hechas usualmente por desarrolladores sobre unidades pequeñas; luego las de integración, centradas en las interfaces y contratos entre componentes; después las de sistema o versión, sobre el producto integrado y con participación de un equipo de testing; por último las de aceptación de usuario, que permiten decidir con el cliente si el incremento satisface lo esperado. En la práctica, la integración suele quedar cerca de desarrollo y producir un *build* para las pruebas de sistema. La aceptación se realiza con intervención del usuario o en su presencia, y puede habilitar la puesta en producción.

Un mínimo razonable contempla tres **ambientes** separados: desarrollo, prueba y producción. El esquema deseable agrega preproducción para realizar aceptación antes de afectar a usuarios reales. La separación entre desarrollo y prueba preserva independencia y permite saber qué versión exacta se está probando mientras desarrollo continúa corrigiendo. Integración puede apoyarse en un servidor automatizado bajo la órbita de desarrollo.

Caja negra observa las entradas y salidas del producto desde sus especificaciones, sin necesitar conocer la implementación. Entre sus métodos se encuentran la **partición de equivalencias**, que selecciona representantes de clases de valores válidos e inválidos, y el **análisis de valores límite**, que examina los bordes de esos rangos. **Caja blanca** usa el conocimiento de la estructura interna para orientar la prueba; comprende criterios de cobertura de sentencias, decisiones y condiciones, así como el análisis de complejidad ciclomática para estimar caminos independientes. Las estrategias son complementarias y sus métodos ayudan a seleccionar casos sin intentar todas las combinaciones posibles.

Dinámico significa ejecutar el software para observar su comportamiento. **Estático** significa analizar un artefacto sin ejecutarlo, por ejemplo al revisar código. Esta distinción describe el modo de análisis y no debe confundirse con las estrategias de caja negra y caja blanca: conocer la estructura interna también puede guiar pruebas ejecutadas.

2. Tipos de prueba y foco de cada caso

Al diseñar una prueba hay que decidir **qué se quiere comprobar**. Los requisitos funcionales y no funcionales son categorías de requisitos del producto; no constituyen, por sí mismos, una lista suficiente de tipos de prueba. “Testing funcional” también se usa como expresión general para el con-

junto de pruebas sobre funcionalidad, e incluso a veces como nombre de la operación normal. Lo importante es poder explicar el objetivo concreto del caso, más allá de la variación en los nombres.

Conviene evitar que un mismo recorrido intente medir objetivos incompatibles. Por ejemplo, si se ingresan varias fechas inválidas para observar validaciones, el tiempo humano invertido en esos intentos no sirve para concluir cuánto tarda el sistema en responder en condiciones normales. Un caso puede reutilizar una secuencia de acciones para distintos fines en ejecuciones separadas, pero necesita datos, condiciones y resultados esperados adecuados para cada objetivo.

Tipo o foco	Qué se comprueba
Operación normal	El uso previsto o “camino feliz”: con entradas válidas, la funcionalidad completa su objetivo y produce el resultado esperado. Es una base indispensable para las historias de usuario.
Prueba negativa	El sistema rechaza datos o acciones indebidas y responde de manera controlada. Por ejemplo, un valor no válido para una fecha debe generar la validación esperada; que se impida esa operación es el resultado exitoso de la prueba.
Proceso de negocio	Un circuito completo que concatena varias funcionalidades o historias. Para inscribirse a cursar pueden intervenir el alta de un alumno, la validación de documentación, la asignación de legajo y la inscripción. Es posible mantener casos separados por funcionalidad y ejecutar el conjunto para cubrir el proceso completo.
Documentación	La ayuda de uso, instalación o configuración debe corresponderse con la versión real del sistema. Un manual puede ser una página, wiki, video o ayuda integrada; su formato no cambia la necesidad de comprobar que las instrucciones y pantallas coincidan.

Pruebas sobre requisitos no funcionales

Los requisitos no funcionales describen **cómo** debe comportarse el producto y bajo qué condiciones. Para comprobarlos se necesitan objetivos medibles, una configuración de entorno conocida y datos de prueba apropiados. Una experiencia satisfactoria en un equipo local no permite inferir automáticamente el desempeño del sistema desplegado para muchos usuarios.

Tipo	Qué se observa o mide
Rendimiento	Tiempo transcurrido entre una solicitud del usuario y la respuesta del sistema; también se puede observar el comportamiento con operaciones concurrentes.
Carga y escalabilidad	Comportamiento frente a una cantidad esperada de usuarios, peticiones o datos, y capacidad de sostener el servicio cuando esa demanda crece. Los límites y metas deben derivarse de los requisitos comprometidos.
Estrés y recuperación	Comportamiento al superar la carga prevista o agotar un recurso: cuándo se degrada o falla el servicio y cuánto tarda en recuperarse una vez retirada la condición extrema.
Usabilidad y accesibilidad	Facilidad con la que las personas comprenden y completan tareas, claridad de la interfaz y posibilidad de uso por personas con distintas capacidades. La experiencia de usuario requiere observación y criterios concretos.
Mantenibilidad	Facilidad y esfuerzo necesarios para corregir el producto o adaptarlo ante un cambio de requisitos sin introducir problemas adicionales.
Fiabilidad	Consistencia y confiabilidad de los resultados y del servicio durante su operación, incluidas situaciones en las que pueden aparecer fallas.
Portabilidad, configuración e instalación	Capacidad de funcionar al trasladarlo o configurarlo en otros entornos compatibles: sistema operativo, red, controladores, componentes y servicios externos; también que pueda instalarse y quedar listo para usar.
Seguridad	Protección frente a accesos o acciones no autorizadas y cumplimiento de las restricciones de acceso previstas. Puede requerir herramientas y escenarios especializados.

Las **pruebas de interfaz de usuario** requieren atención particular porque combinan formas de interacción, navegación, entradas y presentación de resultados. Pueden examinar comportamientos funcionales y propiedades de usabilidad o accesibilidad; el objetivo preciso determina los casos, los datos y el resultado esperado.

Una **prueba de humo** es un chequeo inicial y breve sobre un build para determinar si está en condiciones de recibir pruebas más profundas, por ejemplo si inicia y sus funciones esenciales no presentan una falla catastrófica. No reemplaza las pruebas de sistema completas.

> **Regla y proceso de negocio.** Una regla condiciona cómo debe comportarse el sistema, como los requisitos previos para rendir una materia o una tasa impositiva aplicable. Un proceso de negocio encadena acciones para cumplir un objetivo, como inscribirse a un examen; en su recorrido debe respetar varias reglas.

3. Qué cambia al probar en un equipo ágil

La diferencia principal es de **momento, finalidad y cultura de trabajo**. En un esquema donde las pruebas se concentran después de implementar una versión, el tiempo consumido en requisitos, diseño y programación suele dejar poco margen para probar. Un ciclo tradicional no tiene por qué ser cascada: también puede ser iterativo e incremental. Aun así, si dentro de cada incremento se deja la prueba para el final, aparece el mismo riesgo de recortarla. Los casos pueden prepararse desde que se conocen los requisitos, antes de tener el código ejecutable.

En ágil se intenta completar una historia o porción de valor con análisis, diseño, construcción y pruebas dentro de la iteración. El **Definition of Done** incluye las comprobaciones acordadas: una historia sin ellas no está terminada. Si el tiempo fijo del Sprint se acaba, resulta más útil contar con algunas historias completas que con muchas a medio construir y ninguna lista para entregar. Hacer reuniones diarias o usar el nombre Scrum no modifica por sí mismo una secuencia de “todos los requisitos, luego todo el código, luego todas las pruebas”.

El énfasis se mueve de **encontrar defectos al final** hacia **prevenirlos y recibir feedback temprano**, sin dejar de buscar los que sí se introduzcan. El equipo necesita comprender el dominio de negocio, no limitarse a constatar que un botón funciona. Validar que la solución sea útil importa tanto como verificar su comportamiento técnico. La responsabilidad por la calidad corresponde a todas las personas que construyen el producto; quien prueba hace visibles los problemas y ayuda a evitarlos, no absorbe por sí solo la responsabilidad de toda la calidad.

La **automatización** hace viable repetir comprobaciones con cada cambio y acortar el tiempo entre introducirlo y saber si rompió algo. No todo se automatiza: la exploración, la observación del uso real y ciertas evaluaciones de usabilidad requieren juicio humano. Automatizar lo repetible libera tiempo para ese trabajo.

4. Valores, principios y prácticas del testing ágil

Los valores expresan la dirección general, los principios orientan las decisiones y las prácticas son formas concretas de llevarlas a cabo. Aplicar una práctica aislada no alcanza para instalar una cultura colaborativa de calidad.

Valor	Consecuencia para el equipo
Probar a lo largo del desarrollo	Las pruebas forman parte del flujo de cada historia y no quedan como una fase de cierre.
Prevenir defectos	Se busca detectar problemas y malentendidos antes o apenas se introducen.
Entender lo que se prueba	Se conversan reglas, ejemplos y necesidades del negocio, además de comprobar funciones.
Construir un mejor sistema	El tester coopera con el resto del equipo en vez de limitarse a "romper" el producto.
Compartir la responsabilidad por la calidad	Desarrollo, producto y testing se involucran; la calidad no se delega en un único rol.

Los **nueve principios** proponen: mover el testing hacia el comienzo del trabajo; tratarlo como una actividad continua y no una fase; hacer que todo el equipo participe; reducir la espera hasta recibir feedback; representar las expectativas en las pruebas; mantener el código limpio y corregir defectos pronto; reducir documentación redundante; incluir las pruebas en el Done; y pasar de probar solamente al final a dejar que las pruebas guíen el desarrollo.

Entre las **prácticas concretas** se ubicaron las pruebas unitarias y de integración automatizadas, la regresión automatizada a nivel de sistema, las pruebas exploratorias, TDD, ATDD y el versionado de las pruebas junto con el código. La regresión vuelve a comprobar tanto lo corregido como aquello que antes funcionaba: repetir solo el caso que falló puede dejar escondida una rotura nueva en otra parte.

Reducir documentación no equivale a dejar de registrar lo necesario. Los casos de prueba necesitan precisión para reproducirse. En enfoques como la **especificación mediante ejemplos** o BDD, los ejemplos de comportamiento acordados pueden convertirse en pruebas y funcionar como **documentación viva**. Su utilidad depende de que se mantengan al día: si cambia una regla, hay que actualizar los ejemplos y las pruebas, conservar alineados comportamiento, código y expectativas, y evitar versiones separadas y contradictorias de la misma información.

5. TDD, ATDD y participación del tester

TDD parte de pruebas próximas a la unidad de código y guía su implementación con ciclos cortos. Es una práctica de desarrollo impulsado por pruebas. **ATDD** arranca desde una característica o historia de usuario, explora ejemplos de negocio y acuerda pruebas de aceptación de alto nivel *antes* de programar la solución. A partir de esos ejemplos se derivan pruebas concretas, se automatiza lo que convenga y se implementa hasta satisfacerlas. Ambos adelantan el feedback, pero se enfocan en distintos niveles.

Característica de negocio → historia y ejemplos → criterios y pruebas de aceptación → implementación → comprobación y ajuste

En un equipo ágil, el tester participa desde la comprensión de la historia: ayuda al Product Owner a revisarla y precisar criterios de aceptación, conversa con desarrollo sobre los ejemplos y la automatización, colabora con integración y mantiene las condiciones necesarias para probar. También comprueba que la historia cumpla el Done acordado. Su aporte deja de limitarse a recibir una versión al final y redactar una lista de defectos.

La colaboración no elimina la necesidad de una mirada independiente. Puede ocurrir que una persona implemente una historia y otra la pruebe, o que existan especialistas de prueba dentro del mismo equipo. Además de probar cada historia, hay que pensar en **el incremento integrado**: interacciones entre historias, requisitos no funcionales y condiciones de un release. El equipo debe reservar tiempo dentro de su trabajo para ello y decidir qué pruebas tienen sentido en cada momento. Posponer sistemáticamente todo a un Sprint exclusivo de “endurecimiento” deja historias aparentemente terminadas sin comprobar como incremento.

6. Cuadrantes del testing ágil

Los cuadrantes ordenan las actividades según dos preguntas: **¿orientadas al negocio o a la tecnología?** y **¿ayudan a guiar el desarrollo o critican el producto obtenido?** No establecen fases ni un orden obligatorio. Sirven para advertir si se está cubriendo solo una perspectiva y dejando de lado otras.

Cuadrante	Orientación y ejemplos	Automatización
Q1 Tecnología + guía	Pruebas unitarias y de componentes a nivel de código. Apoyan la calidad interna y dan feedback inmediato al desarrollo.	Muy adecuadas para automatizar e integrar en cada cambio.
Q2 Negocio + guía	Ejemplos, criterios de aceptación y pruebas funcionales que especifican cómo debe comportarse una historia. Pueden usarse prototipos o simulaciones para aclarar expectativas.	Se combina conversación y exploración manual con pruebas automatizables.
Q3 Negocio + crítica	Evaluación desde el uso real: pruebas exploratorias, usabilidad, flujos de trabajo y aceptación para descubrir situaciones no previstas.	El juicio humano es central; lo no anticipado es difícil de automatizar.
Q4 Tecnología + crítica	Rendimiento, carga, estrés, seguridad, recuperación y otros atributos no funcionales del producto integrado.	Se necesitan herramientas y entornos apropiados; por ejemplo, simular muchas solicitudes concurrentes.

Un equipo que automatiza solo Q1 puede comprobar muy rápido el código y, aun así, construir la funcionalidad equivocada. Q2 introduce expectativas de negocio; Q3 ayuda a detectar problemas que no se habían imaginado; Q4 examina propiedades del sistema que no se descubren con un caso funcional aislado. El esfuerzo concreto se decide según riesgos y estado del incremento.

7. Pirámide de pruebas y automatización

La **pirámide** sugiere concentrar muchos chequeos rápidos en la base —unitarios y de componentes —, una cantidad menor en integración y servicios, y menos recorridos completos por interfaz de usuario en la cima. Las pruebas completas siguen siendo necesarias, pero son más lentas y costosas de preparar y sostener. En un enfoque tradicional el esfuerzo suele concentrarse arriba, en pruebas manuales de sistema e interfaz; la propuesta ágil redistribuye el esfuerzo hacia una base automatizada que puede ejecutarse con cada cambio.

Zona	Ejemplo y razón de uso
Base amplia	Reglas y unidades pequeñas: rápidas, numerosas y localizan fallos con precisión.
Centro	Integración entre módulos, controladores, servicios y API: confirma contratos e intercambios.
Cima reducida	Flujos completos en navegador, sistema y aceptación: confirman la experiencia integrada.

La proporción es una **guía de distribución**, no una cuota fija ni una garantía de calidad. Automatizar reduce el tiempo de ejecución, permite reutilizar casos de regresión, ampliar la frecuencia y la cobertura practicable, y dedicar el trabajo humano a explorar riesgos nuevos. Hay que diseñar, implementar y mantener los tests, los datos y los entornos; por eso el ahorro depende de qué se automatiza y de cuántas veces se vuelve a ejecutar. La integración continua ayuda a que cada cambio reciba señales tempranas de las pruebas automatizadas.